Analysis of Finding best Recruiters

November 3, 2024

Sarvan Suthar (2023CSB1159) ,
Vikash Kumar(2023CSB1172) ,
Sumit Singh Bais(2023CSB1166)

Instructor:
Dr. Anil Shukla

Teaching Assistant:
Deepu Yadav

Summary: Our project is designed to facilitate the matching of candidates with suitable recruiters based on the skills of the candidates and the specific demands of the recruiters. To achieve this, we have implemented the trie data structure.The trie data structure allows for efficient storage and retrieval of recruiter and candidate information. It supports fast autocomplete features, enabling users to quickly find relevant recruiters based on partial inputs. This significantly enhances the user experience, as it reduces search times and improves the accuracy of the results.

---

1. Introduction

In today's competitive job market, the process of connecting candidates with suitable recruiters has become increasingly complex. Recruiters seek specific skills and qualifications, while candidates aim to find opportunities that align with their expertise. Our project addresses this challenge by developing an innovative system that leverages advanced data structures to enhance the recruitment process.

The primary objective of our project is to facilitate the matching of candidates with appropriate recruiters based on their skills and the demands of the recruiters. By utilizing a trie data structure, we enable fast and efficient searching capabilities, allowing users to quickly locate recruiters that match their skill sets. The trie not only supports rapid autocomplete features but also helps in categorizing recruiters, making it easier for candidates to navigate the platform.

In this system, candidates can input their skills or areas of expertise, and the trie structure efficiently processes this information to provide relevant recruiter suggestions. This not only streamlines the search process but also increases the chances of candidates connecting with recruiters who are genuinely interested in their profiles.

Ultimately, our project aims to create a user-friendly platform that optimizes the recruitment experience for both candidates and recruiters. By integrating innovative technology with a focus on user needs, we aspire to enhance the efficiency and effectiveness of the recruitment process, paving the way for better job matches in the labor market.

2. Trie(Data Structure)

A trie, also known as a prefix tree or digital tree, is a specialized tree-like data structure that is used to efficiently store and retrieve keys in a dataset of strings. Unlike traditional data structures such as arrays or linked lists, tries are particularly effective for tasks involving string matching, searching, and autocomplete features. The structure is built in such a way that each node represents a character of a key, allowing for common prefixes to

be stored only once, thus saving space.

Fast Search and Insert Operations: The average time complexity for searching, inserting, and deleting keys in a trie is O(m), where m is the length of the key. This efficiency makes tries particularly suitable for applications requiring quick lookups and updates.
Space Efficiency: Although tries can consume more memory than other data structures due to their node-based nature, they optimize storage by sharing common prefixes. For example, the words "cat", "car", and "cart" share the prefix "ca", which is stored only once.

The filtering mechanism in our project leverages the trie data structure to enhance the recruitment process by focusing on three key areas: location, industry, and experience level. For location-based filtering, the trie indexes recruiters according to geographical preferences, enabling candidates to swiftly find recruiters that match their desired locations, thereby saving time and ensuring recruiters connect with interested candidates in specific regions. Industry-specific categorization is also facilitated by the trie, which allows candidates to filter searches based on their industry preferences; this targeted approach leads to efficient matches as candidates receive suggestions for recruiters specializing in their field. Lastly, by integrating experience level matching into the trie, our system allows candidates to specify their qualifications, ensuring they are presented with recruiters seeking candidates within their experience range, ultimately increasing the chances of successful job placements.

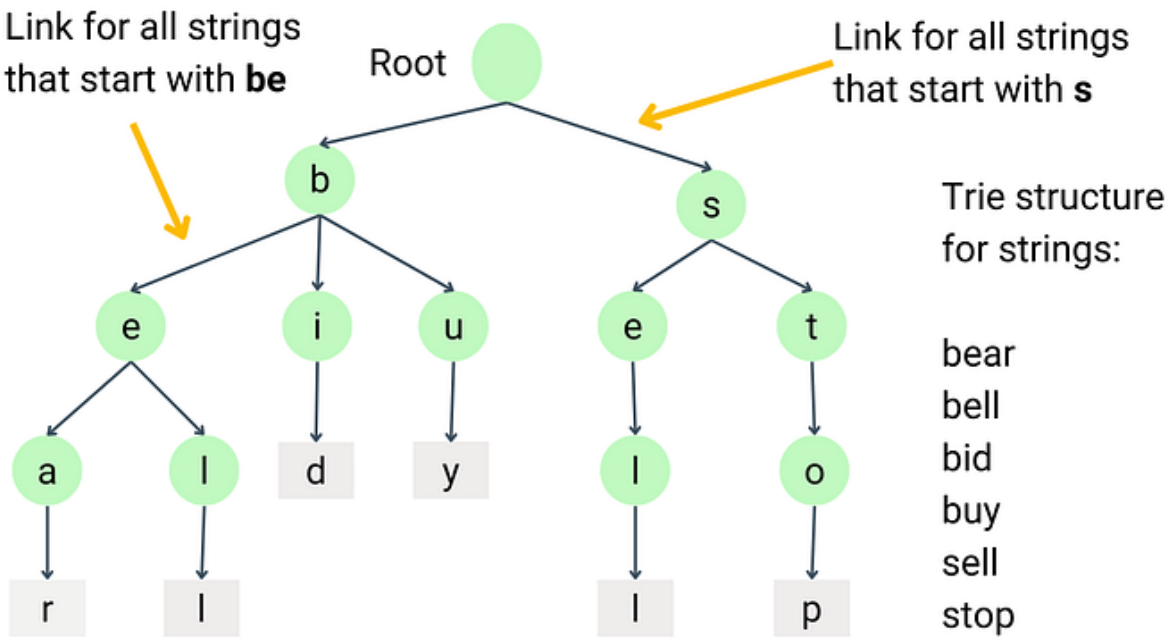

Figure 1: Representation of Trie

2.1. Complexities

	Expected Complexity	Worst Case Complexity
Search	O(m)	O(k)
Insert	O(m)	O(m)

Table 1: m : Length of the string,

k : Length of the currently present longest string

3. Max. Heap (Data Structure)

A max heap is a binary tree data structure that satisfies the heap property, where each parent node has a value greater than or equal to its child nodes. This structure ensures that the largest element is always at the root, making it efficient for implementing priority queues. Max heaps are complete binary trees, meaning all levels are fully filled except possibly the last one, which is filled from left to right.

In our project, we utilized a max heap to prioritize recruiters based on the expected salary in lakhs per annum (LPA). The max heap structure allows us to efficiently manage and retrieve recruiters, ensuring that those offering the highest salaries are readily accessible. By inserting each recruiter into the max heap according to their expected salary, we maintain a dynamic ordering that facilitates quick access to the highest-paying opportunities.

When candidates express interest in salary expectations, the max heap enables us to efficiently extract the top recruiters, aligning candidates with lucrative positions. This prioritization not only streamlines the job search process for candidates but also enhances their chances of securing roles that meet their financial expectations. Overall, the implementation of a max heap in our project significantly contributes to making the recruitment process more effective and responsive to candidate needs.

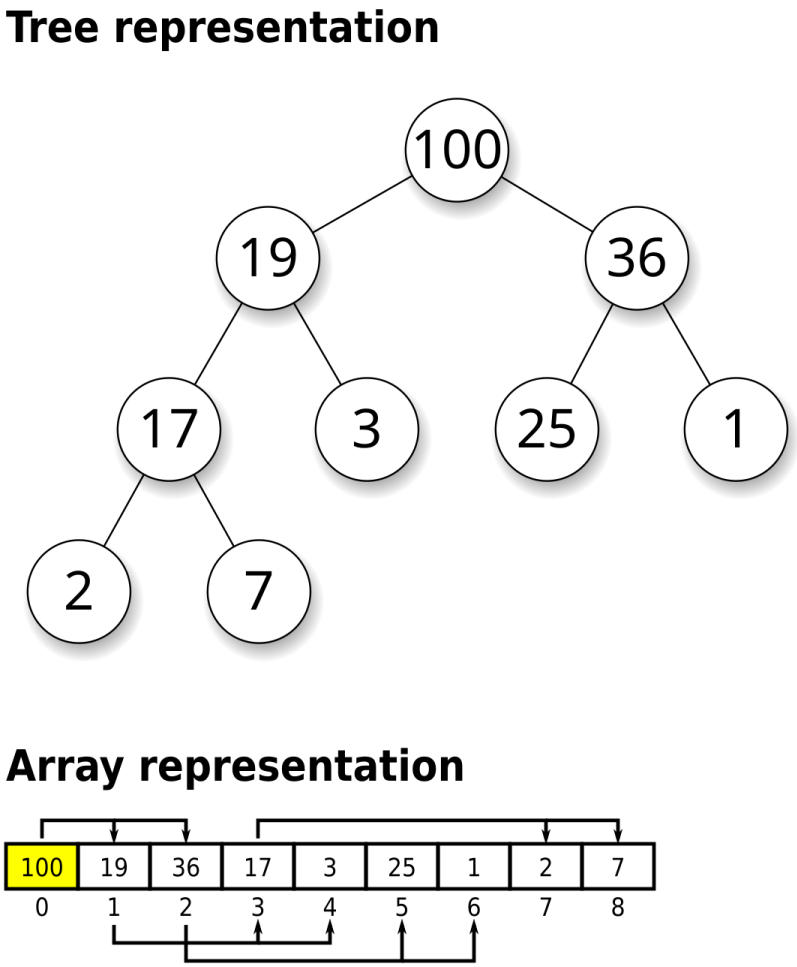

Figure 2: Representation of Max. Heap

3.1. Complexities

	Expected Complexity	Worst Case Complexity
Peek	O(1)	O(1)
Insert	O(logn)	O(logn)
Building Heap	O(n)	O(n)

Table 2: n : Number of element in heap

4. Algorithms

4.1. Trie Construction

The Trie data structure allows for efficient storage and retrieval of recruiter information based on candidate preferences. The construction of the Trie involves initializing a root node and then inserting nodes corresponding to the characters in the recruiter's attributes.

Step 1: Initialization: Create a root node for the trie, which acts as the entry point for all subsequent nodes.

Step 2: Insertion: For each recruiter in the dataset, perform the following:

Sub-step 1: Start from the root node.

Sub-step 2: For each character in the recruiter's location, industry, and experience level:

Item 1: If the character node does not exist, create a new node. This dynamically builds the trie as unique paths are added for each character.

Item 2: Move to the next character node, continuing this process until the entire attribute has been inserted.

Sub-step 3: At the end of the string, mark the node as a terminal node and store recruiter information (like ID or reference) at that node. This allows for the retrieval of all relevant recruiters when a search is conducted.

Algorithm 1 Trie Construction

```
void insertRecruiter(const std::string &industry,const Recruiter&recruiter,string location) {
    std::shared_ptr<TrieNode> node = root;

    for (char ch : industry) {
        if (!node->children.count(ch)) {
            node->children[ch] = std::make_shared<TrieNode>();
        }
        node = node->children[ch];
    }

    node->experienceMap[recruiter.experienceYears][location].push_back(recruiter);
}
```

4.2. Trie Search

The Trie search process enables candidates to find recruiters that match their preferences quickly. By traversing the Trie based on the input criteria, we can efficiently retrieve the relevant recruiters.

To find recruiters based on candidate preferences:

Step 1: Start from the root node and process each character of the input (location, industry, or experience level).

Step 2: Traverse through the trie according to the input characters, checking for the existence of each character node.

Step 3: Upon reaching the end of the input string, retrieve the list of recruiters stored at the terminal node, which corresponds to the input preferences.

Algorithm 2 Trie Search

```
std::vector<Recruiter> findRecruitersByPrefix(const std::string &prefix, int minExperience,
string location="") {
    std::shared_ptr<TrieNode> node = root;
    std::vector<Recruiter> matchingRecruiters;

    for (char ch : prefix) {
        if (!node->children.count(ch)) {
            return matchingRecruiters;
        }
        node = node->children[ch];
    }

    gatherRecruiters(node, minExperience, matchingRecruiters, location);
    return matchingRecruiters;
}
```

4.3. Max Heap for Salary Prioritization

A Max Heap is used to prioritize recruiters based on their expected salaries. This data structure allows for efficient retrieval of the highest salary recruiter, ensuring candidates are presented with the best opportunities.

4.4. Max Heap Construction

The construction of the Max Heap involves inserting recruiters who match the filtered criteria from the Trie. During insertion, the heap property is maintained to ensure that the highest salary recruiter is always accessible.

Step 1: Insert Recruiters: For each recruiter who matches the candidate's filtered criteria from the trie:

Sub-step 1: Insert the recruiter into the max heap using the expected salary as the key, ensuring that recruiters are prioritized based on their compensation.

Sub-step 2: During insertion, compare the salary with the parent node and bubble up if necessary to maintain the max heap property. This guarantees that the highest salary is always at the top of the heap, making it easy to access.

Algorithm 3 Heapify (After insertion)

```
void heapify(int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;
        if (heap[index].first > heap[parent].first) {
            swap(heap[index], heap[parent]);
            index = parent;
        } else {
            break;
        }
    }
}
```

4.5. Extracting Top Recruiters

To provide candidates with the best options, the algorithm retrieves the top N recruiters based on salary from the Max Heap. This ensures that candidates are presented with the most competitive opportunities.
To get the top N recruiters based on salary:

Step 1: Initialize a list to hold the top recruiters, preparing to collect the highest salary matches.

Step 2: While the max heap is not empty and the required number of recruiters hasn't been retrieved:

Sub-step 1: Extract the maximum (top recruiter) from the max heap. This operation provides access to the recruiter with the highest expected salary.

Sub-step 2: Add the extracted recruiter to the list of top recruiters, accumulating the best matches based on salary.

Step 3: Repeat the extraction until the desired count is reached or the heap is empty, ensuring that the output contains the top N recruiters that fit the candidate's criteria.

Algorithm 4 Extracting Top Recruiters

```
pair<double, Recruiter> pop() {
    if (heap.empty()) {
        throw out_of_range("Heap is empty");
    }
    pair<double, Recruiter> maxElement = heap[0];
    heap[0] = heap.back();
    heap.pop_back();
    heapifyDown(0);
    return maxElement;
}
```

4.6. Overall Algorithm Flow

This section outlines the complete workflow of the algorithm, detailing how candidate preferences are collected, recruiters are filtered, and top opportunities are presented.

5. Conclusions

This project develops an efficient recruitment system using a Trie for filtering and a Max Heap for salary prioritization, ensuring candidates quickly find optimal job opportunities. The approach streamlines the recruitment process and enhances candidate experience..

Possible future developments of this project would be

1. Enhanced Search Features
2. Expanded Data Sources
3. Mobile Application Development

6. Applications

1. Helps candidates find job opportunities that match their location, industry, and experience preferences.

2. Promotes job openings and employer brands to targeted audiences

3. Showcases company culture and values to attract aligned candidates.

7. Bibliography and citations

Acknowledgements

We extend our heartfelt gratitude to our instructor, Dr. Anil Shukla, and our Teaching Assistant, Deepu Yadav, for their invaluable guidance and insightful contributions throughout the project.

References

[1] Edward Fredkin. Trie data structures. *Communications of the ACM*, pages 490–498, 1960.

[2] David Jones. *Heap Structures and Their Efficiency in Algorithm Design*. PhD thesis, University of Waterloo, 2015.

[3] Jainam M. Max heap and its applications. Technical report, parallelcomputing2017.wordpress.com, 2018.

[4] Wikipedia. Trie. Technical report, Wikipedia, 2023.